

CSE33 I: Microprocessor Interfacing and Embedded Systems

Lecture#22: Inter-Integrated Circuit (I2C)

Chapter 22

Summer, 2023

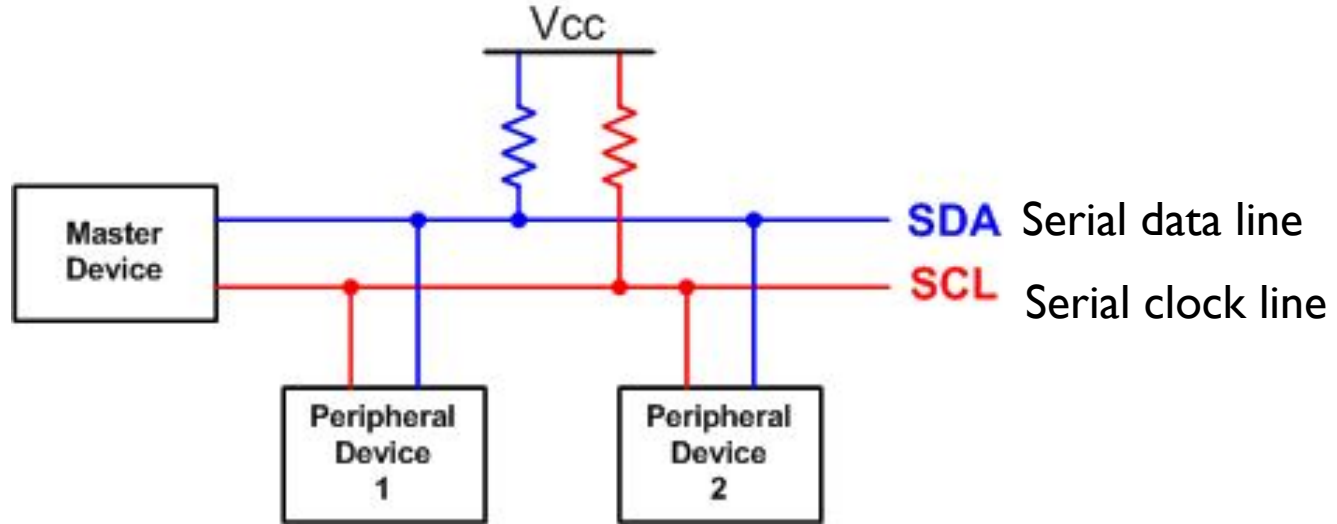
Mohammad A. Qayum, Ph.D.
ECE, NSU, Dhaka, Bangladesh

Inter-Integrated Circuit (I2C)

- Designed for low-cost, medium data rate applications by Philips in the early 1980's
 - Original purpose: connect a CPU to peripheral chips in a TV-set
 - Today: a de-facto standard for 2-wire communications
 - Since October 10, 2006, no licensing fees are required to implement the I²C protocol. However, fees are still required to obtain I²C slave addresses allocated by NXP (acquired Philips).

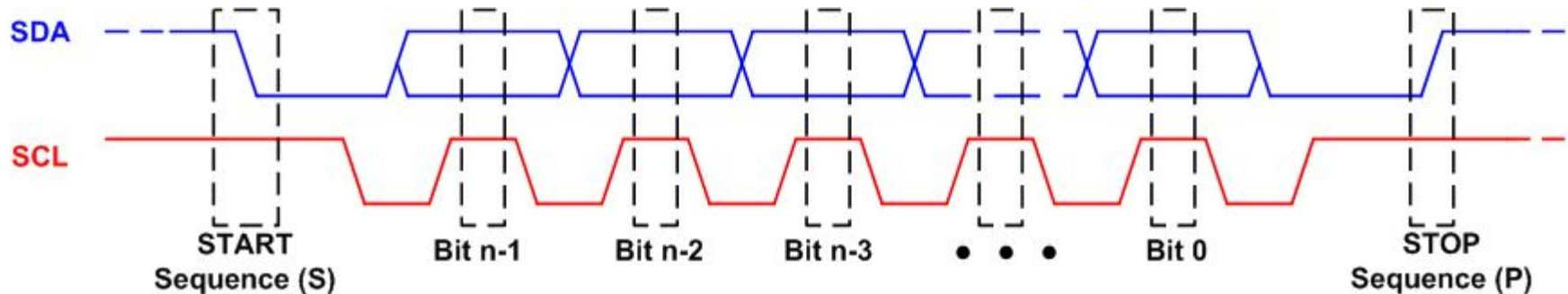
- Characteristics
 - Serial, byte-oriented
 - Multi-master, multi-slave
 - Two bidirectional open-drain lines, plus ground
 - Serial Data Line (SDA)
 - Serial Clock Line (SCL)
 - SDA and SCL need to pull up with resistors

Inter-Integrated Circuit (I2C)



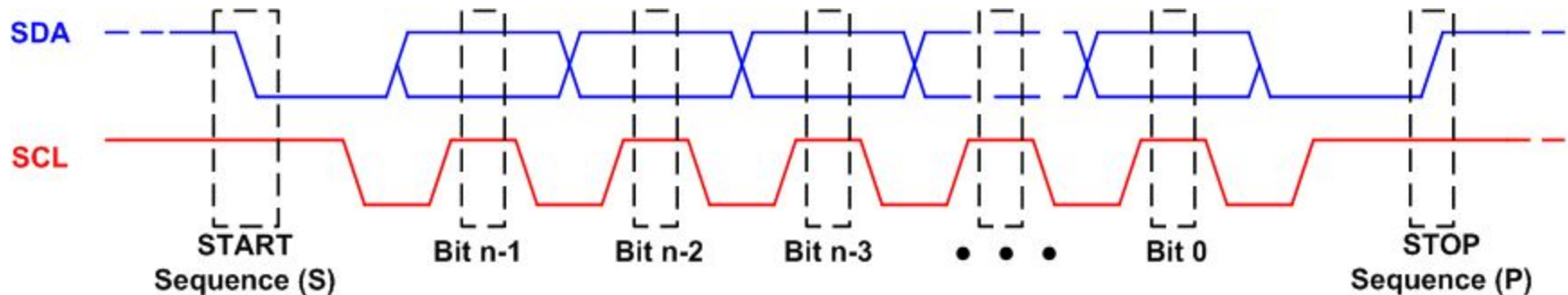
- Up to 100 kbit/s in the standard mode, up to 400 kbit/s in the fast mode, and up to 3.4 Mbit/s in the high-speed mode.
- SDA and SCL have to be open-drain
 - Connected to positive if the output is 1
 - In high impedance state if the output is 0
- Each Device has an unique address (7, 10 or 16 bits). Address 0 used for broadcast
- STM32L's internal pull-up is too weak (internal 100K Ω)
- External pull-up (4.7 k Ω for low speed, 3 k Ω for standard mode, and 1 k Ω for fast mode).

Timing Diagram



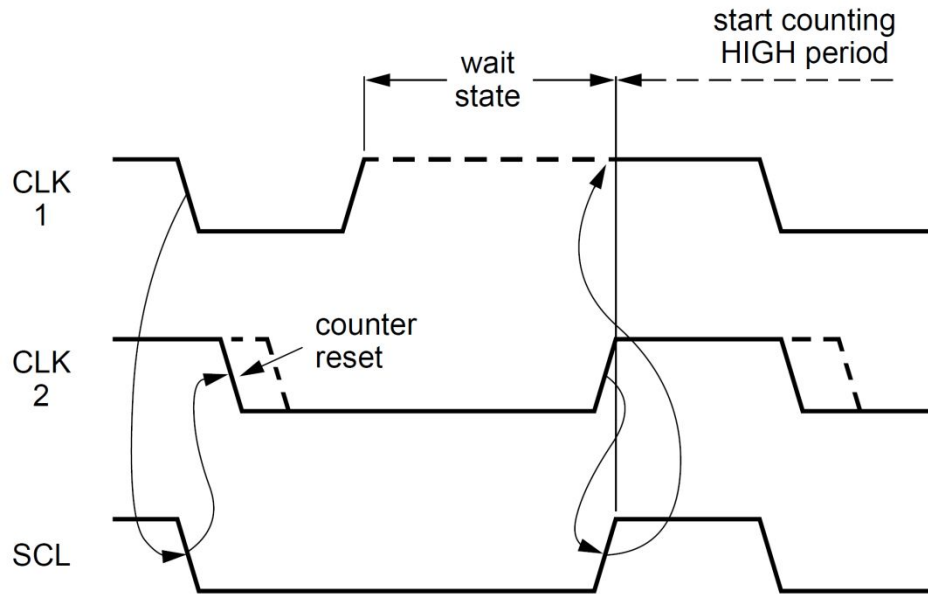
- A **START** condition is a high-to-low transition on SDA when SCL is high.
- A **STOP** condition is a low to high transition on SDA when SCL is high.
- The address and the data bytes are sent most significant bit first.
- Master generates the clock signal and sends it to the slave during data transfer

Multiple Masters



- “Wired-AND” bus: A sender can pull the lines to low, even if other senders are trying to drive the lines to high
- In single master systems, arbitration is not needed.
- Arbitration for multiple masters:
 - During data transfer, the master constantly checks whether the SDA voltage level matches what it has sent.
 - When two masters generate a START setting concurrently, the first master which detects SDA low while it has actually intended to set SDA high will lose the arbitration and let the other master complete the data transfer.

Clock Synchronization



Source: I2C Specifications

- Clock synchronization is needed when there are multiple masters.
- **Wired-AND** connection for clock synchronization
 - Each master has a counter. Counter resets if SCL goes LOW. When the counter counts down to zero, the master releases SCL and thus SCL goes high.
 - SCL remains LOW if any master pulls it LOW.
 - When all masters concerned have counted off their LOW period, the clock line is released and goes HIGH.
 - After going high, all masters start counting their HIGH periods. The first master to complete its HIGH period pulls the SCL line LOW again.

Working Modes

□ Master-sender □

- Master issues START and ADDRESS, and then transmits data to the addressed slave device

□ Master-receiver □

- Master issues START and ADDRESS, and receives data from the addressed slave device

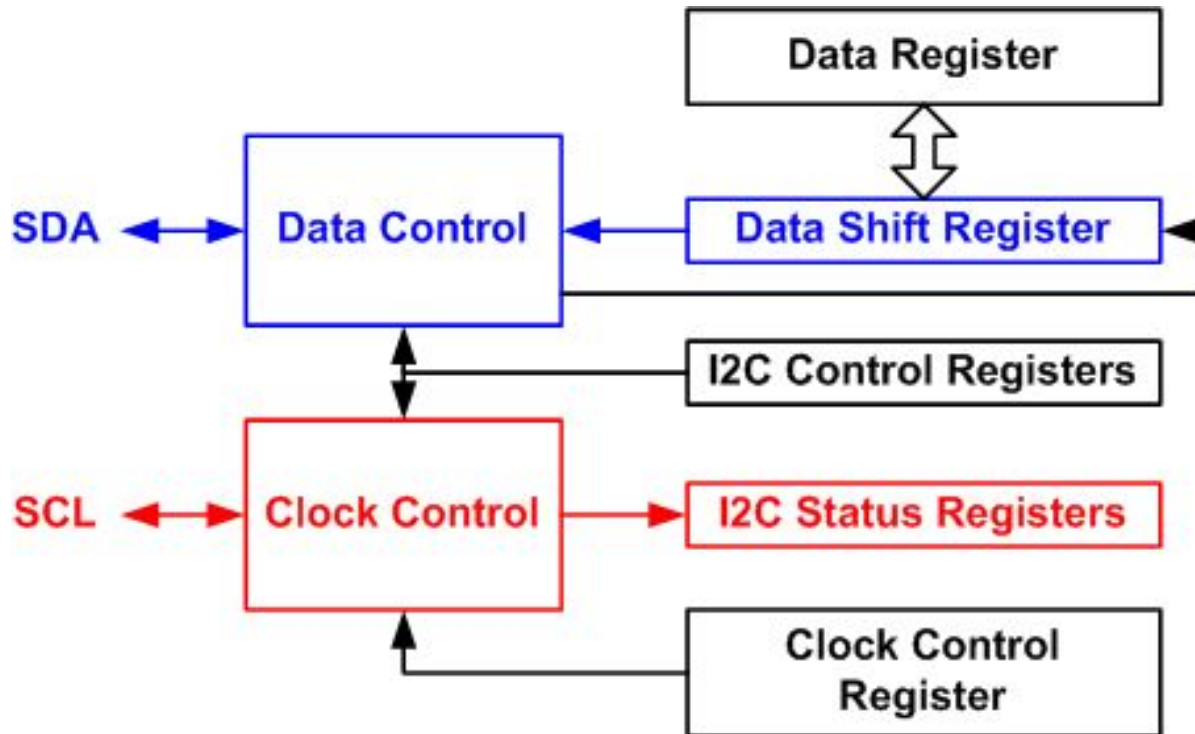
□ Slave-sender □

- Master issues START and the ADDRESS of the slave, and then the slave sends data to the master

□ Slave-receiver □

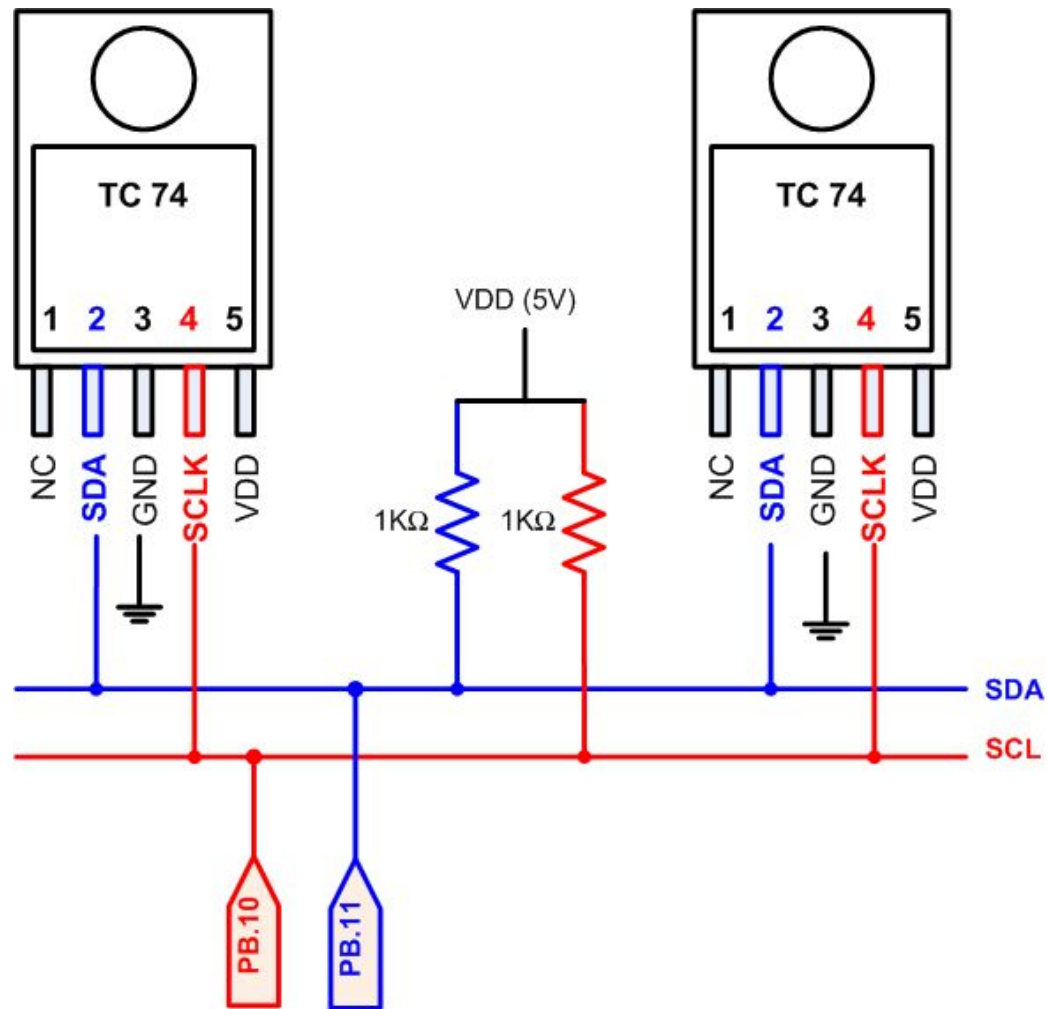
- Master issues START and the ADDRESS of the slave, and then the slave receives data from the master.

STM32L I2C Module

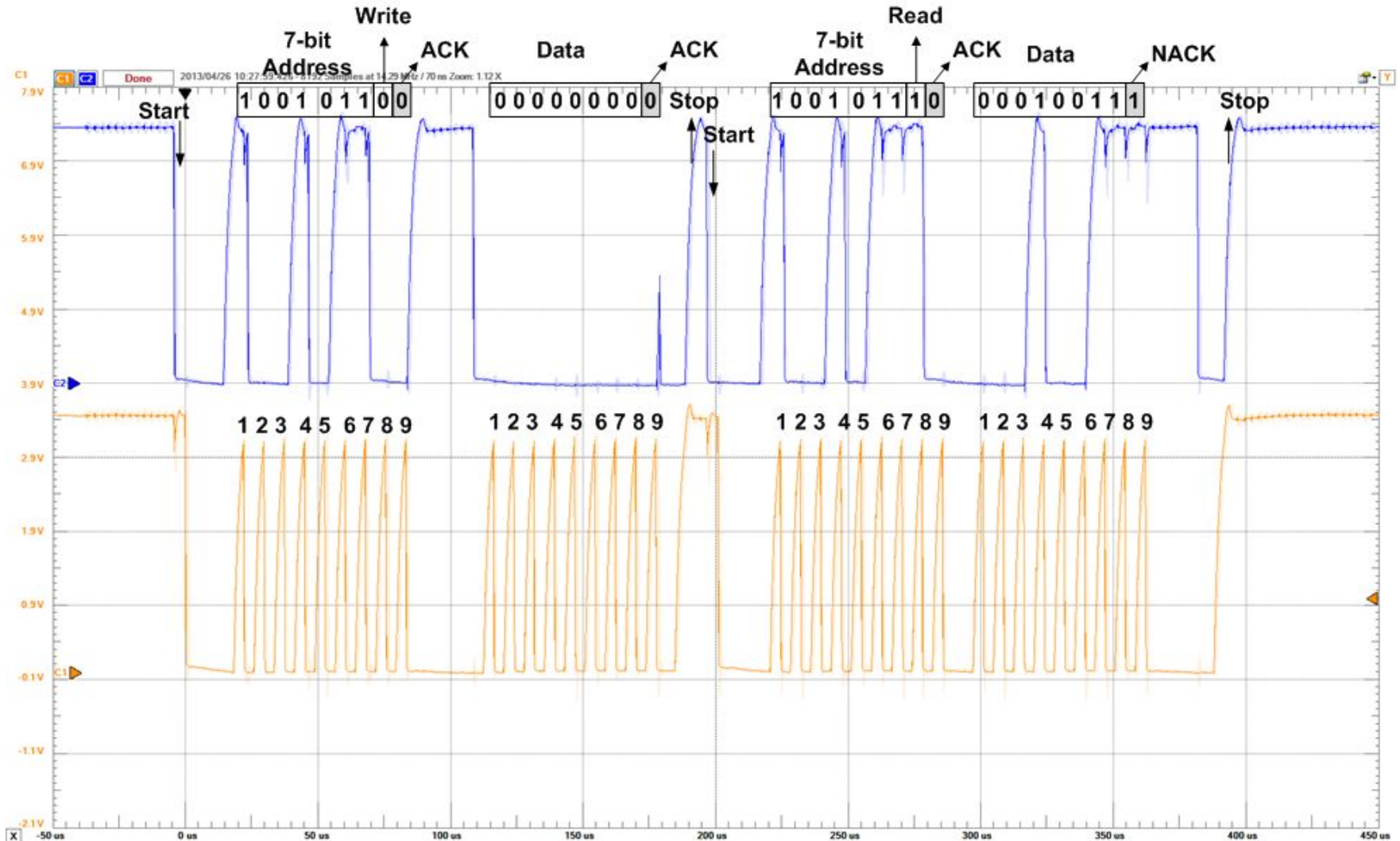


- During sending, the I²C hardware automatically sets the **TxE flag** in the status register if an acknowledge pulse is received from the slave.
- During receiving, the I²C hardware then automatically sets the **RxNE flag** in the status register if a byte has been successfully received.

Interfacing Serial Digital Thermal Sensor



I2C Data



I2C Initialization

```
void I2C_Initialization(void){
    // Enable the clock of I2C
    RCC->APB1ENR  |= RCC_APB1ENR_I2C2EN;          // I2C 2 clock enable

    // Reset the I2C (set and reset by software)
    RCC->APB1RSTR |= RCC_APB1RSTR_I2C2RST;
    RCC->APB1RSTR &= ~RCC_APB1RSTR_I2C2RST;

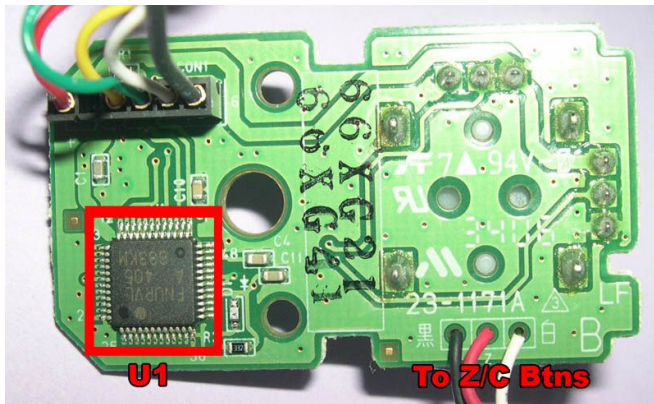
    // I2C Control register. ACK is set and cleared by software, and cleared by hardware when PE=0.
    I2C2->CR1 |= I2C_CR1_ACK;          // Acknowledge after a byte is received
    I2C2->CR1 &= ~I2C_CR1_SMBUS;       // SMBus Mode: 0 = I2C mode; 1 = SMBus mode
    I2C2->CR2 &= ~I2C_CR2_FREQ;        // Clear Peripheral CLK frequency FREQ[5:0]
    I2C2->CR2 |= I2C_CR2_FREQ_1;        // Set Peripheral clock frequency as 2 MHz
    I2C2->CR2 |= I2C_CR2_ITEV TEN;       // Event Interrupt Enable
    I2C2->CR1 &= ~I2C_CR1_PE;          // Disable I2C to configure TRISE
    I2C2->TRISE &= ~I2C_TRISE_TRISE;    // Clear Maximum Rise Time
    I2C2->TRISE |= 17;                  // Set Maximum Rise Time for standard mode

    // I2C own address registers
    // Before the STM32 sends its start sequence it sits listening to the I2C lines waiting for its address
    // This is helpful if STM32 is used as slave
    I2C2->OAR1 %= ~(0x01);

    I2C2->OAR1 = 0x00;    // Set up the address byte to select the slave device
    I2C2->OAR2 = 0x00;    // Set up the I2C own address2

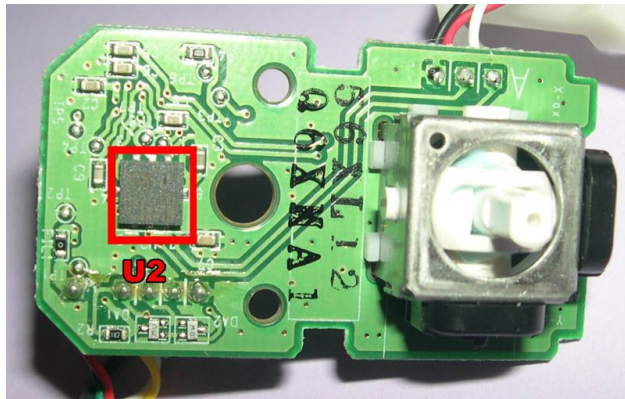
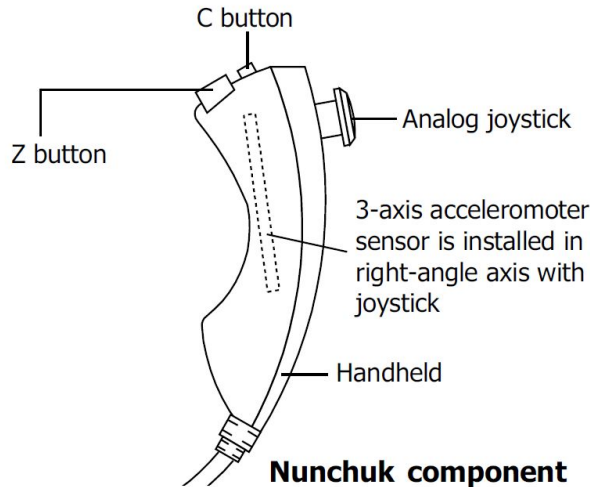
    // I2C Clock control register
    I2C2->CCR &= ~I2C_CCR_FS;    // 0 = Standard Mode ((up to 100 kHz));
                                // 1 = Fast Mode (up to 400 kHz)
    I2C2->CCR &= ~I2C_CCR_CCR;
    I2C2->CCR |= 0x08;
    I2C2->CR1 |= I2C_CR1_PE;    // Enable I2C1
}
```

Wii Nunchuck



- 3 Axis accelerometer.
- Can capture X,Y and Z motion
- STMicroelectronics **LIS3L02AL**
 - 3.0 V to 3.6 V supply voltage
 - ~2 mW power consumption
 - ± 2 g full-scale
 - 3 acceleration channels plus multiplexed analog output
 - Ratiometric output voltage
 - Power-down mode
 - Embedded self-test
 - 10000 g high shock survivability

Wii Nunchuck



Offset	Data							
0x00	Joystick X							
0x01	Joystick Y							
0x02	Accelerometer X (bit 9 to bit 2)							
0x03	Accelerometer Y (bit 9 to bit 2)							
0x04	Accelerometer Z (bit 9 to bit 2)							
0x05	Accel. Z (bit 1)	Accel. Z (bit 0)	Accel. Y (bit 1)	Accel. Y (bit 0)	Accel. X (bit 1)	Accel. X (bit 0)	C button	Z button

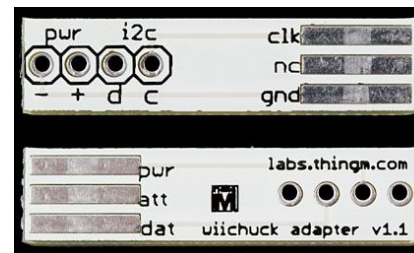
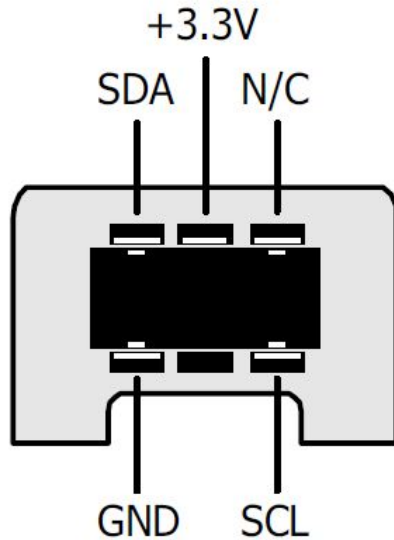
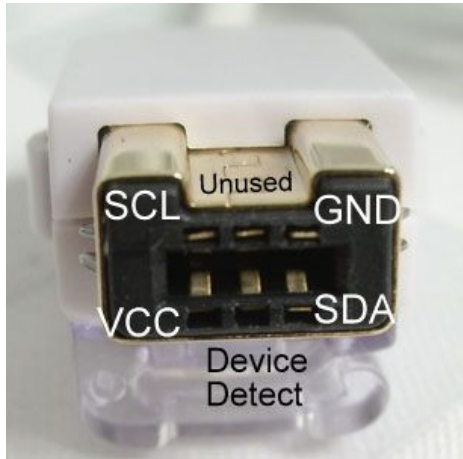
- Two push buttons (labeled as C and Z)
- An 8-bit 2-axis analog joystick (X,Y) and
- A 10-bit 3-axis accelerometer sensor (X,Y, and Z)
- Communication is encrypted.
- One possible decoding is

$$\text{Data} = (\text{Received data XOR } 0x17) + 0x17$$

Joystick X	0x80 = Center, 0x00 = Full left, 0xFF = Full right
Joystick Y	0x80 = Center, 0x00 = Full up, 0xFF = Full down
Acceleration	0 – 1023.
Button	0 = pressed, 1 = released

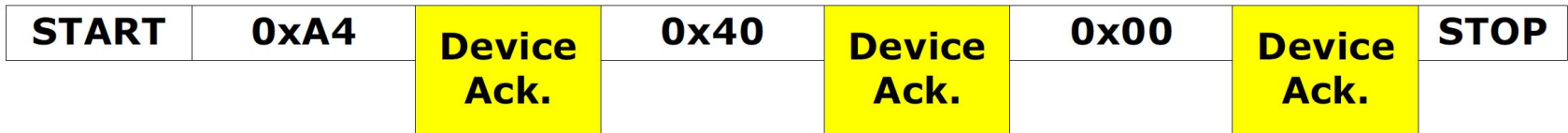
□ I2C Standard Mode (100Kbps)

Interfacing Wii Nunchuck



Nunchuck Commands

Initialization Command

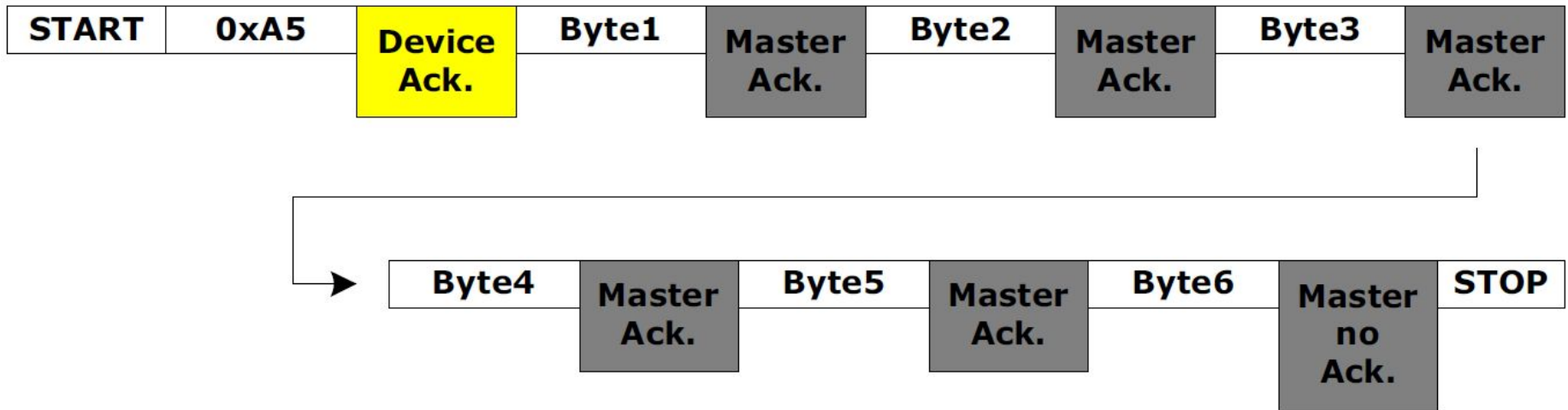


Conversion Command



- Each Nunchuck has Two Slave Addresses
 - Slave command address: 0xA4
 - Slave data address: 0xA5

Nunchuck Read Data



Offset	Data							
0x00	Joystick X							
0x01	Joystick Y							
0x02	Accelerometer X (bit 9 to bit 2)							
0x03	Accelerometer Y (bit 9 to bit 2)							
0x04	Accelerometer Z (bit 9 to bit 2)							
0x05	Accel. Z (bit 1)	Accel. Z (bit 0)	Accel. Y (bit 1)	Accel. Y (bit 0)	Accel. X (bit 1)	Accel. X (bit 0)	C button	Z button

Left and Right

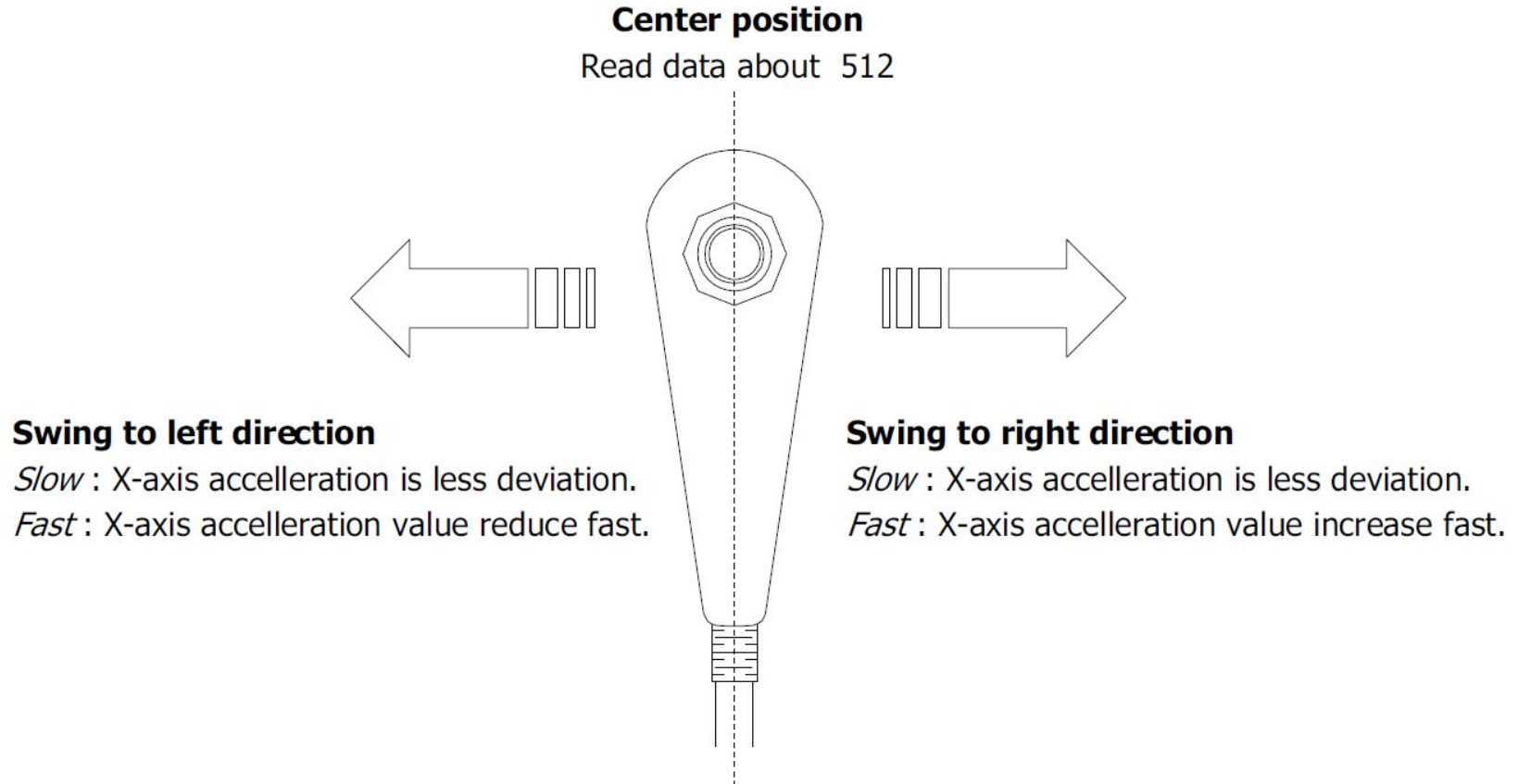


Image from ZX-NUMCHUK
(#800339)

Forward and Backward

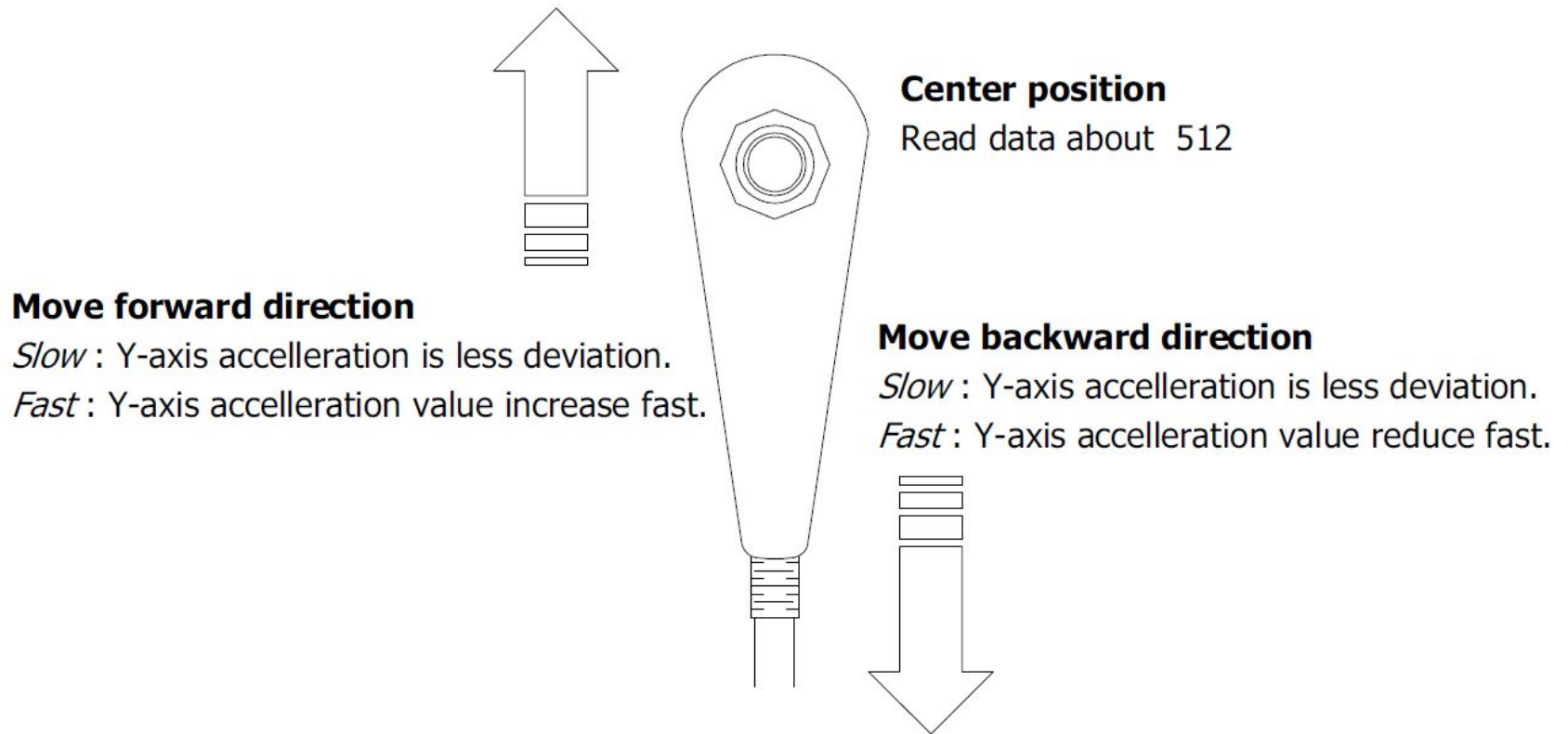


Image from ZX-NUMCHUK
(#800339)

Up and Down

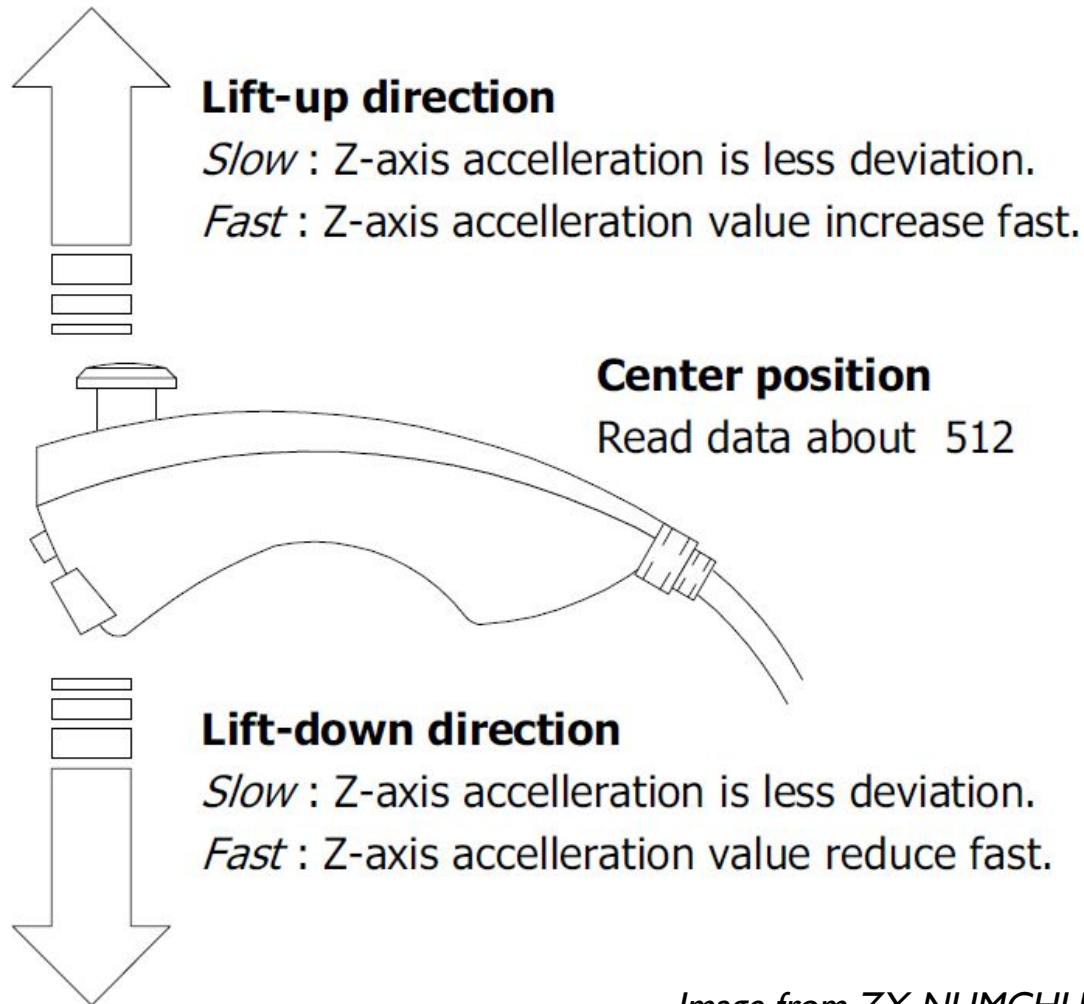


Image from ZX-NUMCHUK
(#800339)

References

- Materials are adopted from:
 - ZX-NUNCHUK Wii-Nunchuk interface board
 - Chad - Wii Nunchuk/Wiichuck Adapter